04. 제약 조건 시스템

목차

1. 개요
2. FlexGate: 기본 게이트
3. GateInstructions 트레이트
4. RangeGate: 범위 검사 게이트
5. RangeInstructions 트레이트
6. 관련 문서

개요

halo2-lib의 제약 조건 시스템은 영지식증명 회로에서 계산의 정확성을 보장하는 핵심 메커니즘입니다. 이 문서에서는 두 가지 주요 게이트 시스템을 다룹니다:

- **FlexGate**: 기본 산술 연산(덧셈, 곱셈 등)을 위한 커스텀 게이트
- **RangeGate**: lookup 테이블을 사용한 범위 검사 게이트

이 두 게이트는 각각 `GateInstructions` 와 `RangeInstructions` 트레이트를 통해 고수준 API를 제공합니다.

FlexGate: 기본 게이트

BasicGateConfig 구조

FlexGate의 핵심은 `BasicGateConfig` 구조체입니다. 이 구조체는 단일 advice 컬럼과 selector를 사용하여 기본 산술 연산을 구현합니다.

소스: `halo2-base/src/gates/flex_gate/mod.rs`

```
#[derive(Clone, Debug)]
pub struct BasicGateConfig<F: ScalarField> {
/// Selector 컬럼: 게이트 활성화 여부를 제어
pub q_enable: Selector,
/// Advice 컬럼: 게이트의 값들을 저장
pub value: Column<Advice>,
_marker: PhantomData<F>,
}
```

Vertical Gate Strategy

FlexGate는 "Vertical Gate Strategy"를 사용합니다. 이는 하나의 제약 조건으로 다양한 연산을 표현하는 전략입니다:

제약 조건 공식:

```
q * (a + b * c - d) = 0
```

여기서:

- `a = value[0]` (현재 행)
- `b = value[1]` (다음 행)
- `c = value[2]` (2번째 다음 행)
- `d = value[3]` (3번째 다음 행)

- `q = q_enable[0]` (selector, 0 또는 1)

이 단일 제약 조건으로 다양한 연산을 표현할 수 있습니다:

- **덧셈**: `a + b = d` (c = 1로 설정)
- **곱셈**: `b * c = d` (a = 0으로 설정)
- **곱셈-덧셈**: `a + b * c = d`

create_gate() 메서드

`create_gate()` 메서드는 위의 제약 조건을 실제로 회로에 추가합니다:

```
fn create_gate(&self, meta: &mut ConstraintSystem<F>) {
meta.create_gate("1 column a + b * c = out", |meta| {
let q = meta.query_selector(self.q_enable);
// 4개의 연속된 행에서 값을 가져옴
let a = meta.query_advice(self.value, Rotation::cur());
let b = meta.query_advice(self.value, Rotation::next());
let c = meta.query_advice(self.value, Rotation(2));
let out = meta.query_advice(self.value, Rotation(3));
// 제약 조건: q * (a + b * c - out) = 0
vec![q * (a + b * c - out)]
})
}
```

FlexGateConfig::configure()

FlexGateConfig 는 여러 phase에 걸쳐 여러 개의 BasicGateConfig 를 관리합니다:

```
pub fn configure(meta: &mut ConstraintSystem<F>, params: FlexGateConfigParams)
// Fixed 컬럼 생성 (상수 값용)
let mut constants = Vec::with_capacity(params.num_fixed);
for _i in 0..params.num_fixed {
let c = meta.fixed_column();
meta.enable_equality(c);
constants.push(c);
}
// 각 phase별로 BasicGateConfig 생성
let mut basic_gates = vec![];
for (phase, &num_columns) in params.num_advice_per_phase.iter().enumerate() {
let config = (0..num_columns)
.map(|_| BasicGateConfig::configure(meta, phase as u8))
.collect();
basic_gates.push(config);
}

Self {
basic_gates,
constants,
max_rows: (1 << params.k) - meta.minimum_rows(),
```

```
}

}
```

GateInstructions 트레이트

`GateInstructions` 트레이트는 FlexGate를 사용한 고수준 산술 연산 API를 제공합니다. `GateChip` 이 이 트레이트를 구현합니다.

add() 메서드: 덧셈

두 값을 더하는 연산: `a + b = out`

```
fn add(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
) -> AssignedValue<F> {
let a = a.into();
let b = b.into();
let out_val = *a.value() + b.value();
// 게이트: | a | b | 1 | a + b |
ctx.assign_region_last([a, b, Constant(F::ONE), Witness(out_val)], [0])
}
```

사용 예시:

```rust
let gate = GateChip::default();
let ctx = builder.main(0);

let a = ctx.load_witness(F::from(5));
let b = ctx.load_witness(F::from(3));
let sum = gate.add(ctx, a, b); // sum = 8
```

sub() 메서드: 뺄셈

두 값의 차를 계산: `a - b = out`

```rust
fn sub(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
) -> AssignedValue<F> {
let a = a.into();
let b = b.into();
let out_val = *a.value() - b.value();
// 게이트: | a - b | b | 1 | a |
ctx.assign_region([Witness(out_val), b, Constant(F::ONE), a], [0]);
ctx.get(-4)
```

```
}
```

사용 예시:

```
let a = ctx.load_witness(F::from(10));
let b = ctx.load_witness(F::from(3));
let diff = gate.sub(ctx, a, b); // diff = 7
```

mul() 메서드: 곱셈

두 값을 곱하는 연산: `a * b = out`

```
fn mul(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
) -> AssignedValue<F> {
let a = a.into();
let b = b.into();
let out_val = *a.value() * b.value();
// 게이트: | 0 | a | b | a * b |
ctx.assign_region_last([Constant(F::ZERO), a, b, Witness(out_val)], [0])
}
```

사용 예시:

```
let a = ctx.load_witness(F::from(4));
let b = ctx.load_witness(F::from(7));
let product = gate.mul(ctx, a, b); // product = 28
```

inner_product() 메서드: 내적

두 벡터의 내적을 계산: `<a, b> = a[0]*b[0] + a[1]*b[1] + ... + a[n-1]*b[n-1]`

```
fn inner_product<QA>(
&self,
ctx: &mut Context<F>,
a: impl IntoIterator<Item = QA>,
b: impl IntoIterator<Item = QuantumCell<F>>,
) -> AssignedValue<F>
where
QA: Into<QuantumCell<F>>,
{
self.inner_product_simple(ctx, a, b);
ctx.last().unwrap()
}
```

내부적으로 `inner_product_simple` 은 누적 합을 계산하며, `b` 가 `Constant(F::ONE)` 으로 시작하는 경우 최적화를 수행합니다.

사용 예시:

```
let a = vec![
ctx.load_witness(F::from(1)),
ctx.load_witness(F::from(2)),
ctx.load_witness(F::from(3)),
];
let b = vec![
Constant(F::from(4)),
Constant(F::from(5)),
Constant(F::from(6)),
];

// 내적: 1*4 + 2*5 + 3*6 = 4 + 10 + 18 = 32
let result = gate.inner_product(ctx, a.clone(), b.into_iter().map(|x| x));
```

select() 메서드: 조건부 선택

selector 값에 따라 두 값 중 하나를 선택: `sel ? a : b`

```
fn select(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
```

```
sel: impl Into<QuantumCell<F>>,

) -> AssignedValue<F> {

let  a = a.into();

let  b = b.into();

let  sel = sel.into();

let  diff_val = *a.value() - b.value();

let  out_val = diff_val * sel.value() + b.value();

// out = sel * a + (1 - sel) * b

// out = sel * (a - b) + b

let  cells = [

Witness(diff_val),

Constant(F::ONE),

b,

a,

b,

sel,

Witness(diff_val),

Witness(out_val),

];

ctx.assign_region_smart(cells, [0, 4], [(0, 6), (2, 4)], []);

ctx.last().unwrap()

}
```

사용 예시:

```
let a = ctx.load_witness(F::from(10));
let b = ctx.load_witness(F::from(20));
let sel = ctx.load_witness(F::ONE); // true

let result = gate.select(ctx, a, b, sel); // result = 10 (a가 선택됨)

let sel_false = ctx.load_witness(F::ZERO); // false
let result2 = gate.select(ctx, a, b, sel_false); // result2 = 20 (b가 선택됨)
```

is_equal() 메서드: 동등성 검사

두 값이 같은지 확인하고, 같으면 1, 다르면 0을 반환

```
fn is_equal(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
) -> AssignedValue<F> {
let diff = self.sub(ctx, a, b);
self.is_zero(ctx, diff)
}
```

`is_zero` 메서드는 값이 0인지 확인합니다:

```
fn is_zero(&self, ctx: &mut Context<F>, a: AssignedValue<F>) -> AssignedValue<F

let x = a.value();

let (is_zero, inv) = if x.is_zero_vartime() {

(F::ONE, Assigned::Trivial(F::ONE))

} else {

(F::ZERO, Assigned::Rational(F::ONE, *x))

};

// a가 0이면 is_zero = 1, 아니면 is_zero = 0

// inv는 a의 역원 (a != 0일 때)

let cells = [

Witness(is_zero),

Existing(a),

WitnessFraction(inv),

Constant(F::ONE),

Constant(F::ZERO),

Existing(a),

Witness(is_zero),

Constant(F::ZERO),

];

ctx.assign_region_smart(cells, [0, 4], [(0, 6)], []);

ctx.get(-2)
```

```
}
```

사용 예시:

```
let a = ctx.load_witness(F::from(5));
let b = ctx.load_witness(F::from(5));
let c = ctx.load_witness(F::from(7));

let eq1 = gate.is_equal(ctx, a, b); // eq1 = 1 (같음)
let eq2 = gate.is_equal(ctx, a, c); // eq2 = 0 (다름)
```

기타 유용한 메서드들

mul_add(): `a * b + c` 계산

```
let result = gate.mul_add(ctx, a, b, c); // result = a * b + c
```

div_unsafe(): `a / b` 계산 (b ≠ 0 가정)

```
let quotient = gate.div_unsafe(ctx, a, b); // quotient = a / b
```

and(), or(), xor(), not(): 불리언 연산

```
let and_result = gate.and(ctx, a, b); // a && b
let or_result = gate.or(ctx, a, b); // a || b
let xor_result = gate.xor(ctx, a, b); // a ^ b
```

```
let not_result = gate.not(ctx, a); // !a
```

RangeGate: 범위 검사 게이트

RangeConfig 구조

`RangeConfig` 는 lookup 테이블을 사용하여 값이 특정 범위 내에 있는지 효율적으로 검사합니다.

소스: `halo2-base/src/gates/range/mod.rs`

```
#[derive(Clone, Debug)]
pub struct RangeConfig<F: ScalarField> {
/// 기본 게이트 설정
pub gate: FlexGateConfig<F>,
/// 각 phase별 lookup용 특수 advice 컬럼들
pub lookup_advice: Vec<Vec<Column<Advice>>>,
/// Lookup 테이블용 selector들
pub q_lookup: Vec<Option<Selector>>,
/// Lookup 테이블 컬럼
pub lookup: TableColumn,
/// Lookup 테이블이 표현하는 비트 수 [0, 2^lookup_bits)
lookup_bits: usize,
}
```

Lookup 테이블 메커니즘

RangeGate는 lookup 테이블을 사용하여 범위 검사를 수행합니다. Lookup 테이블은 `[0, 2^lookup_bits)` 범위의 모든 값을 포함합니다.

예를 들어, `lookup_bits = 8` 이면 테이블은 0부터 255까지의 값을 포함합니다.

lookup_advice 컬럼의 역할

`lookup_advice` 컬럼들은 범위 검사가 필요한 값들을 저장하는 특수 advice 컬럼입니다. 이 컬럼의 모든 값은 자동으로 lookup 테이블과 대조됩니다.

- **Phase 0에서 advice 컬럼이 1개인 경우**: selector를 사용하여 선택적으로 lookup 활성화
- **그 외의 경우**: 별도의 `lookup_advice` 컬럼 사용 (selector 없이 항상 lookup)

load_lookup_table() 메서드

회로에 lookup 테이블을 로드합니다:

```
pub fn load_lookup_table(&self, layouter: &mut impl Layouter<F>) -> Result<()

layouter.assign_table(

|| format!("{} bit lookup", self.lookup_bits),

|mut table| {

// 0부터 2^lookup_bits - 1까지의 모든 값을 테이블에 할당

for idx in 0..(1u32 << self.lookup_bits) {

table.assign_cell(

|| "lookup table",

self.lookup,

idx as usize,

|| Value::known(F::from(idx as u64)),

)?;
```

```
}
Ok(())
},
)?;
Ok(())
}
```

create_lookup() 메서드

Lookup 제약 조건을 생성합니다:

```
fn create_lookup(&self, meta: &mut ConstraintSystem<F>) {
// Selector를 사용하는 경우
for (phase, q_l) in self.q_lookup.iter().enumerate() {
if let Some(q) = q_l {
meta.lookup("lookup", |meta| {
let q = meta.query_selector(*q);
let a = meta.query_advice(
self.gate.basic_gates[phase][0].value,
Rotation::cur()
);
vec![(q * a, self.lookup)]
});
}
```

```
}

// lookup_advice 컬럼들에 대한 lookup (selector 없음)

for la in self.lookup_advice.iter().flat_map(|advices| advices.iter()) {

meta.lookup("lookup wo selector", |meta| {

let a = meta.query_advice(*la, Rotation::cur());

vec![(a, self.lookup)]

});

}

}
```

RangeInstructions 트레이트

`RangeInstructions` 트레이트는 범위 검사 관련 고수준 API를 제공합니다. `RangeChip` 이 이 트레이트를 구현합니다.

lookup_bits의 역할

`lookup_bits` 는 lookup 테이블의 크기를 결정합니다:

- `lookup_bits = 8` : 테이블 크기 = 256 (0~255)
- `lookup_bits = 16` : 테이블 크기 = 65536 (0~65535)

더 큰 `lookup_bits` 값은:

- **장점**: 더 적은 lookup 횟수로 큰 범위 검사 가능
- **단점**: 더 큰 테이블 크기, 더 많은 메모리 사용

range_check() 메서드: 범위 검사

값이 `[0, 2^range_bits)` 범위 내에 있는지 검사합니다.

```
fn range_check(&self, ctx: &mut Context<F>, a: AssignedValue<F>, range_bits: us
self._range_check(ctx, a, range_bits);
}
```

내부 구현 `_range_check`:

```
fn _range_check(
&self,
ctx: &mut Context<F>,
a: AssignedValue<F>,
range_bits: usize,
) -> AssignedValue<F> {
if range_bits == 0 {
self.gate.assert_is_const(ctx, &a, &F::ZERO);
return a;
}
// limb 개수 계산
let num_limbs = (range_bits + self.lookup_bits - 1) / self.lookup_bits;
let rem_bits = range_bits % self.lookup_bits;
if num_limbs == 1 {
// 단일 limb: 직접 lookup
self.add_cell_to_lookup(ctx, a);
```

```
a
} else {
// 여러 limb: 값을 분해하여 각 limb를 lookup
let limbs = decompose_fe_to_u64_limbs(a.value(), num_limbs, self.lookup_bits)
.into_iter()
.map(|x| Witness(F::from(x)));
let row_offset = ctx.advice.len() as isize;
// 내적을 사용하여 limb들을 재조합
let acc = self.gate.inner_product(
ctx,
limbs,
self.limb_bases[..num_limbs].to_vec()
);
// 재조합된 값이 원래 값과 같은지 확인
ctx.constrain_equal(&a, &acc);
// 각 limb를 lookup 테이블에 추가
self.add_cell_to_lookup(ctx, ctx.get(row_offset));
for i in 0..num_limbs - 1 {
self.add_cell_to_lookup(ctx, ctx.get(row_offset + 1 + 3 * i as isize));
}
let last_limb = ctx.get(row_offset + 1 + 3 * (num_limbs - 2) as isize);
// 마지막 limb에 대한 추가 제약 (rem_bits != 0인 경우)
if rem_bits > 1 {
```

```
let mult_val = self.gate.pow_of_two[self.lookup_bits - rem_bits];
let check = self.gate.mul(ctx, last_limb, Constant(mult_val));
self.add_cell_to_lookup(ctx, check);
} else if rem_bits == 1 {
self.gate.assert_bit(ctx, last_limb);
}
last_limb
}
}
```

사용 예시:

```
let range = builder.range_chip();
let ctx = builder.main(0);

let a = ctx.load_witness(F::from(255));

// a가 8비트 범위 [0, 256) 내에 있는지 검사
range.range_check(ctx, a, 8);

// a가 64비트 범위 [0, 2^64) 내에 있는지 검사
let b = ctx.load_witness(F::from(1000000u64));
range.range_check(ctx, b, 64);
```

check_less_than() 메서드: 비교 연산

`a < b` 인지 확인합니다 (a와 b가 모두 `num_bits` 비트 이하라고 가정).

```
fn check_less_than(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
num_bits: usize,
) {
let a = a.into();
let b = b.into();
let pow_of_two = self.gate.pow_of_two[num_bits];
// check_cell = a + 2^num_bits - b
let check_cell = {
let shift_a_val = pow_of_two + a.value();
let cells = [
Witness(shift_a_val - b.value()),
b,
Constant(F::ONE),
Witness(shift_a_val),
Constant(-pow_of_two),
Constant(F::ONE),
```

```
a,
];
ctx.assign_region(cells, [0, 3]);
ctx.get(-7)
};
// check_cell이 num_bits 범위 내에 있는지 확인
// a < b이면 check_cell < 2^num_bits
self.range_check(ctx, check_cell, num_bits);
}
```

원리:

- `a < b` 를 확인하려면 `a + 2^num_bits - b < 2^num_bits` 를 확인
- 이는 `a + 2^num_bits - b` 가 `num_bits` 비트 범위 내에 있는지 확인하는 것과 동일

사용 예시:

```
let  a = ctx.load_witness(F::from(10));
let  b = ctx.load_witness(F::from(20));

// a < b 확인 (둘 다 8비트 이하)
range.check_less_than(ctx, a, b, 8);
```

is_less_than() 메서드: 불리언 비교

`a < b` 이면 1을, 아니면 0을 반환합니다.

```
fn  is_less_than(
&self,
ctx: &mut Context<F>,
a: impl Into<QuantumCell<F>>,
b: impl Into<QuantumCell<F>>,
num_bits: usize,
) -> AssignedValue<F> {
let  a = a.into();
let  b = b.into();
let  k = (num_bits + self.lookup_bits - 1) / self.lookup_bits;
let  padded_bits = k * self.lookup_bits;
let  pow_padded = self.gate.pow_of_two[padded_bits];
let  shift_a_val = pow_padded + a.value();
let  shifted_val = shift_a_val - b.value();
let  shifted_cell = {
ctx.assign_region(
[
Witness(shifted_val),
b,
Constant(F::ONE),
Witness(shift_a_val),
Constant(-pow_padded),
```

```
Constant(F::ONE),
a,
],
[0, 3],
);
ctx.get(-7)
};
// shifted_cell을 padded_bits + lookup_bits 비트로 범위 검사
let last_limb = self._range_check(ctx, shifted_cell, padded_bits + self.lookup_
// last_limb이 0이면 a < b
self.gate.is_zero(ctx, last_limb)
}
```

사용 예시:

```
let a = ctx.load_witness(F::from(10));
let b = ctx.load_witness(F::from(20));
let c = ctx.load_witness(F::from(5));

let result1 = range.is_less_than(ctx, a, b, 8); // result1 = 1 (10 < 20)
let result2 = range.is_less_than(ctx, a, c, 8); // result2 = 0 (10 >= 5)

// 조건부 로직에 사용
```

```
let selected = gate.select(ctx, x, y, result1); // result1이 1이면 x, 아니면 y
```

기타 유용한 메서드들

check_less_than_safe(): 자동으로 범위 검사를 수행한 후 비교

```
// a < 1000인지 확인 (자동으로 적절한 비트 수 계산)
range.check_less_than_safe(ctx, a, 1000u64);
```

div_mod(): 나눗셈과 나머지 계산

```
// a = b * c + r을 만족하는 (c, r) 반환
let (quotient, remainder) = range.div_mod(ctx, a, b, a_num_bits);
```

decompose_le(): 값을 여러 limb로 분해

```
// num을 8비트 limb 4개로 분해 (little-endian)
let limbs = range.decompose_le(ctx, num, 8, 4);
// limbs[0] = num의 하위 8비트
// limbs[1] = num의 다음 8비트
// ...
```

limbs_to_num(): limb들을 하나의 값으로 재조합

```
// limb들을 다시 하나의 값으로 조합
let num = range.limbs_to_num(ctx, &limbs, 8);
```

전체 사용 예제

다음은 FlexGate와 RangeGate를 함께 사용하는 완전한 예제입니다:

```
use halo2_base::{
gates::{GateChip, GateInstructions, RangeChip, RangeInstructions},
utils::ScalarField,
Context,
QuantumCell::Constant,
};

fn example_circuit<F: ScalarField>(
ctx: &mut Context<F>,
gate: &impl GateInstructions<F>,
range: &impl RangeInstructions<F>,
) {
// 1. 기본 산술 연산
let a = ctx.load_witness(F::from(10));
let b = ctx.load_witness(F::from(20));
let sum = gate.add(ctx, a, b); // sum = 30
let product = gate.mul(ctx, a, b); // product = 200
let diff = gate.sub(ctx, b, a); // diff = 10
// 2. 범위 검사
range.range_check(ctx, a, 8); // a는 8비트 범위 내
```

```
range.range_check(ctx, b, 8); // b는 8비트 범위 내

// 3. 비교 연산

let  is_less = range.is_less_than(ctx, a, b, 8); // is_less = 1

// 4. 조건부 선택

let  x = ctx.load_witness(F::from(100));

let  y = ctx.load_witness(F::from(200));

let  selected = gate.select(ctx, x, y, is_less); // selected = 100 (a < b이므로)

// 5. 내적 계산

let  vec_a = vec![

ctx.load_witness(F::from(1)),

ctx.load_witness(F::from(2)),

ctx.load_witness(F::from(3)),

];

let  vec_b = vec![

Constant(F::from(4)),

Constant(F::from(5)),

Constant(F::from(6)),

];

let  dot_product = gate.inner_product(

ctx,

vec_a.clone(),

vec_b.into_iter()

); // dot_product = 1*4 + 2*5 + 3*6 = 32
```

```
// 6. 동등성 검사
let c = ctx.load_witness(F::from(10));
let is_equal = gate.is_equal(ctx, a, c); // is_equal = 1 (둘 다 10)
// 7. 불리언 연산
let bool_a = ctx.load_witness(F::ONE);
let bool_b = ctx.load_witness(F::ZERO);
let and_result = gate.and(ctx, bool_a, bool_b); // and_result = 0
let or_result = gate.or(ctx, bool_a, bool_b); // or_result = 1
}
```

관련 문서

- 02. 핵심 개념: Context, AssignedValue, QuantumCell 이해
- 03. 회로 구성 프로세스: BaseCircuitBuilder 사용법
- 05. 가상 영역 관리: 제약 조건이 물리적 회로로 매핑되는 과정

다음 단계

제약 조건 시스템을 이해했다면, 이제 가상 영역이 물리적 회로로 매핑되는 과정을 학습하세요:

→ **05. 가상 영역 관리**: Virtual Region Manager와 가상-물리 매핑 프로세스

참고 자료

- **소스 파일**:

- `halo2-base/src/gates/flex_gate/mod.rs` : FlexGate 구현
- `halo2-base/src/gates/range/mod.rs` : RangeGate 구현
- `halo2-base/src/gates/mod.rs` : 게이트 모듈 진입점